

Security Audit

Intiva Health (Utility Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	23
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Intiva Health team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Intiva Health has built a decentralized compliance and credentialing ecosystem for licensed medical professionals through its Incredible platform, which utilizes DLT frameworks to secure credentials and automate regulatory workflows. The platform integrates the Intiva Token (TIVA) to reward verified actions—like document uploads, workflow completion, and ongoing compliance—while also enabling in-app services and subscription discounts. TIVA's incentive model is designed for long-term sustainability through structured reward tiers and a transaction-fee mechanism that supports ecosystem growth.

Project Name: Intiva Health

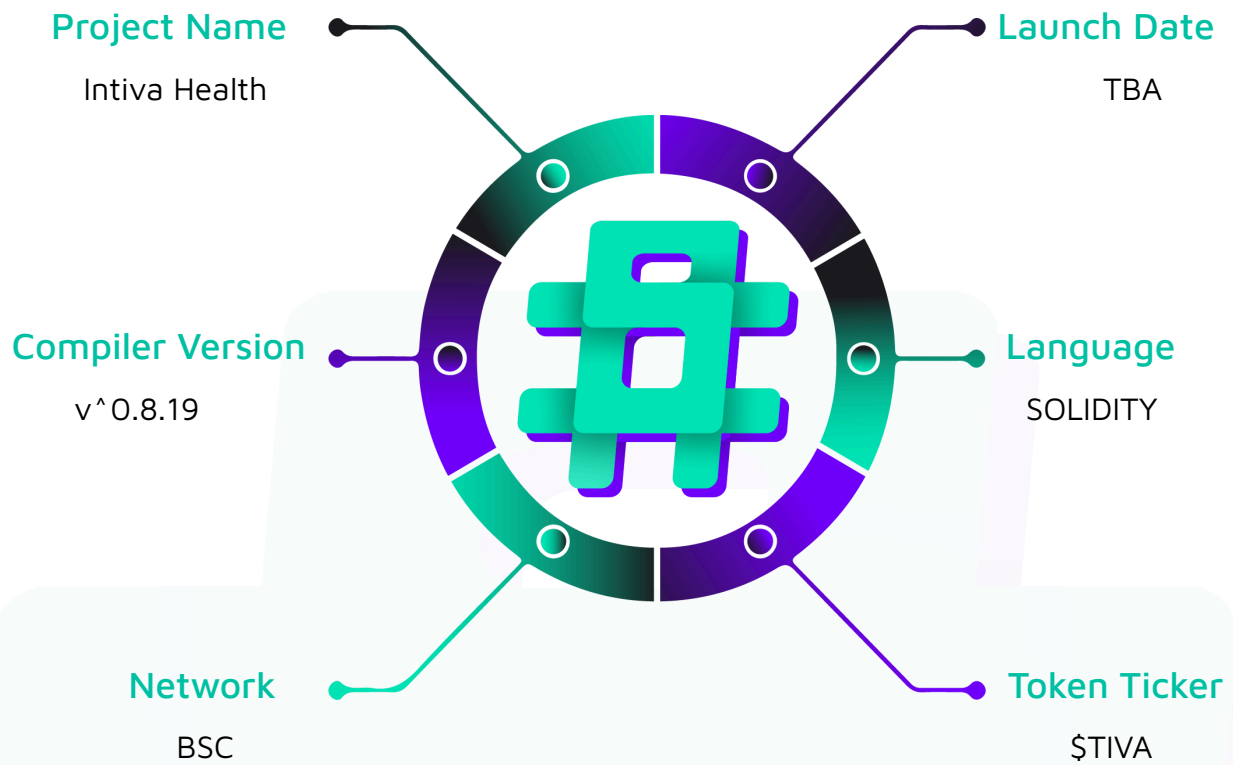
Project Type: Utility Token, dApp.

Compiler Version: ^0.8.19

Website: <https://www.token.intivahealth.com/>

Logo:



Visualised Context:

Project Visuals:



EMPOWERING TRUST IN HEALTHCARE



Now integrated with Incredible, TIVA connects our healthcare community, professionals and supporters alike, with the most trusted credentialing platform in the industry.



TIVA is the digital utility that connects and rewards every part of the Intiva Health Group of Companies. Within Incredible, TIVA drives credentialing, compliance, and engagement through blockchain-based rewards and transactions. It extends directly to patients and providers within the Nova Vita Wellness Centers, and to the future of Gummi World and Droppels, where TIVA can be used to purchase nutraceuticals and OTC products, supporting overall wellness globally. Together, these applications position TIVA as the common engine aligning healthcare technology, wellness services, and product manufacturing within a single, interoperable ecosystem.

Audit Scope

We at Hashlock audited the solidity code within the Intiva Health project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Intiva Health Smart Contracts
Platform	Ethereum / Solidity
Audit Date	November, 2025
Contract 1	NTVAToken.sol
Contract Address	0x57b13A7E18D7382f2CE754ff112b2f552230b83E

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

1 Medium-severity vulnerability

1 Low severity vulnerability

1 Gas Optimisation

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
NTVAToken.sol Act as an ERC20 token with transfer taxation for the Intiva (TIVA) ecosystem Allow users to: - Use the <code>transferTiva()</code> function to transfer with automatic fee deduction Manage tokenomics through: - Fixed supply of 1 billion TIVA tokens minted at deployment - No minting, burning, or pausing capabilities post-deployment - Immutable maximum fee cap of 10% (1000 basis points) Handle fees through: - Configurable fee rate up to MAX_FEE_BPS (1000 bps = 10%) - Automatic fee collection sent directly to the treasury address - Fee calculation in basis points for precision (10000 = 100%) Provide safety features: - Owner-controlled fee adjustment within safe limits - Treasury address updateability for operational flexibility - Zero-address validation on critical parameters - Event emission for transparency on fee and treasury changes	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Intiva Health project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Intiva Health project smart contract code in the form of Bscscan Explorer.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] IntivaToken#transferTiva - Public fee-bearing path lets users unknowingly burn funds

Description

The dedicated `transferTiva` helper was built for Intiva's internal accounting flows, but it remains callable by absolutely anyone. External users who do not realize the difference between `transfer` and `transferTiva` can send tokens through this path and lose up to 10% of the amount because the function silently diverts the fee to the treasury. There is no guardrail, warning, or ABI separation to stop wallets, explorers, or integrators from surfacing this function to the public, so users can grief themselves simply by choosing the wrong transfer helper.

Vulnerability Details

The `IntivaToken` contract introduces a custom `transferTiva` function that properly deducts fees:

```
function transferTiva(address to, uint256 value) public virtual returns(bool) {
    address from = _msgSender();
    uint256 fee = (value * feeBps) / BPS_DENOM;
    if (fee == 0) {
        _transfer(from, to, value);
        return true;
    }
    uint256 amountAfterFee = value - fee;
    _transfer(from, treasury, fee);
    _transfer(from, to, amountAfterFee);
    return true;
}
```

Because the function is public, every ERC-20 wallet, explorer ABI page, or exchange integration automatically exposes it alongside `transfer`. Nothing in the code distinguishes “internal platform operators” from random token holders, so anyone can unknowingly trigger the fee path just by clicking the wrong button in a UI. The function signature also mirrors the standard ERC-20 transfer shape, which makes it especially easy to confuse.

Proof of Concept

An example of a test that simulates a user being grieved through the `transferTiva` function is as follows:

```
contract TransferTivaGriefTest is Test {
    NTVAToken token;
    address owner = address(0x1);
    address treasury = address(0x2);
    address user = address(0x3);
    address recipient = address(0x4);

    function setUp() public {
        token = new NTVAToken(treasury, 1000, owner); // 10% fee
        vm.prank(owner);
        token.transfer(user, 100 ether);
    }

    function testUserAccidentallyBurnsFee() public {
        vm.prank(user);
        token.transferTiva(recipient, 100 ether);

        assertEq(token.balanceOf(recipient), 90 ether, "Recipient only gets 90");
        assertEq(token.balanceOf(treasury), 10 ether, "Treasury captures fee");
    }
}
```

Impact

Any outside user can grief themselves by mistakenly calling `transferTiva`, believing it is the normal transfer helper. For example, sending 100 tokens with a 10% fee results in only 90 tokens reaching the recipient while 10 tokens are siphoned to the treasury, and the caller has no way to recover the deducted amount. This is a guaranteed, silent loss vector that can affect anyone interacting with the ABI directly or through UIs that surface every public method.

Recommendation

Either gate `transferTiva` behind `onlyOwner`/role-based access so only platform-controlled callers can reach it, or convert it to `internal` and expose the internal workflow through a separate contract that never enters the public ABI. If documenting the distinction is preferred, add explicit warnings in the token docs and explorer metadata so outside users immediately understand that `transferTiva` is an internal accounting hook and not a user-facing transfer method

Note

The Intiva Protocol comments that the `transferTIVA` function exists primarily for internal system operations related to recirculating tax calculations. It is strongly encouraged that users not utilize this function for transactions, as it will apply a tax fee—currently 0.25%, with the possibility of increasing to up to 10%.

Standard ERC-20 functions, `transfer` and `transferFrom`, should be used for non platform users as they do not incur any tax and are the preferred, cost-efficient way to transact.

Status

Acknowledged

Low

[L-01] IntivaToken#renounceOwnership -Owner can permanently lock all admin functions

Description

If the contract owner calls `renounceOwnership()`, all administrative functions become permanently inaccessible, preventing updates to fee parameters and treasury address.

Vulnerability Details

The IntivaToken contract inherits OpenZeppelin's `Ownable` contract, which includes a `renounceOwnership()` function:

```
function renounceOwnership() public virtual onlyOwner {  
    _transferOwnership(address(0));  
}
```

The contract has two admin `setFeeBps` and `setTreasury` functions that rely on the `onlyOwner` modifier.

```
function setFeeBps(uint16 newFeeBps) external onlyOwner {...}  
function setTreasury(address newTreasury) external onlyOwner {...}
```

Impact

If `renounceOwnership()` is called, the contract becomes ownerless, and all `onlyOwner` functions are permanently locked, preventing any fee changes, treasury updates, or emergency admin actions.

Recommendation

Override the `renounceOwnership()` function to prevent accidental or malicious renunciation:

```
function renounceOwnership() public view override onlyOwner {  
    revert("Ownership cannot be renounced");  
}
```

Status

Acknowledged



Gas

[G-01] IntivaToken - String-based require statements inflate runtime gas

Description

Using string-based require statements causes unnecessary gas consumption during reverts; custom errors are cheaper and reduce bytecode size.

Vulnerability Details

The `setFeeBps`, `setTreasury`, and `constructor` functions guard inputs with string messages, which takes more gas than custom reverts.

Impact

String-based require statements increase gas costs on every revert, inflate bytecode size for deployment, and waste blockchain resources compared to custom errors.

Recommendation

Define reusable custom errors and replace string-based require:

```
error FeeAboveMax(uint16 attempted);
error TreasuryZeroAddress();
error OwnerZeroAddress();

constructor(address treasury_, uint16 initialFeeBps, address initialOwner)
    ERC20("Intiva Token", "TIVA")
    Ownable(initialOwner)
{
    if (treasury_ == address(0)) revert TreasuryZeroAddress();
    if (initialOwner == address(0)) revert OwnerZeroAddress();
    if (initialFeeBps > MAX_FEE_BPS) revert FeeAboveMax(initialFeeBps);

    treasury = treasury_;
    feeBps = initialFeeBps;
    _mint(initialOwner, 1000000000 * 10**decimals());
}
```

```
}  
  
function setFeeBps(uint16 newFeeBps) external onlyOwner {  
    if (newFeeBps > MAX_FEE_BPS) revert FeeAboveMax(newFeeBps);  
    emit FeeUpdated(feeBps, newFeeBps);  
    feeBps = newFeeBps;  
}  
  
function setTreasury(address newTreasury) external onlyOwner {  
    if (newTreasury == address(0)) revert TreasuryZeroAddress();  
    address old = treasury;  
    treasury = newTreasury;  
    emit TreasuryUpdated(old, newTreasury);  
}
```

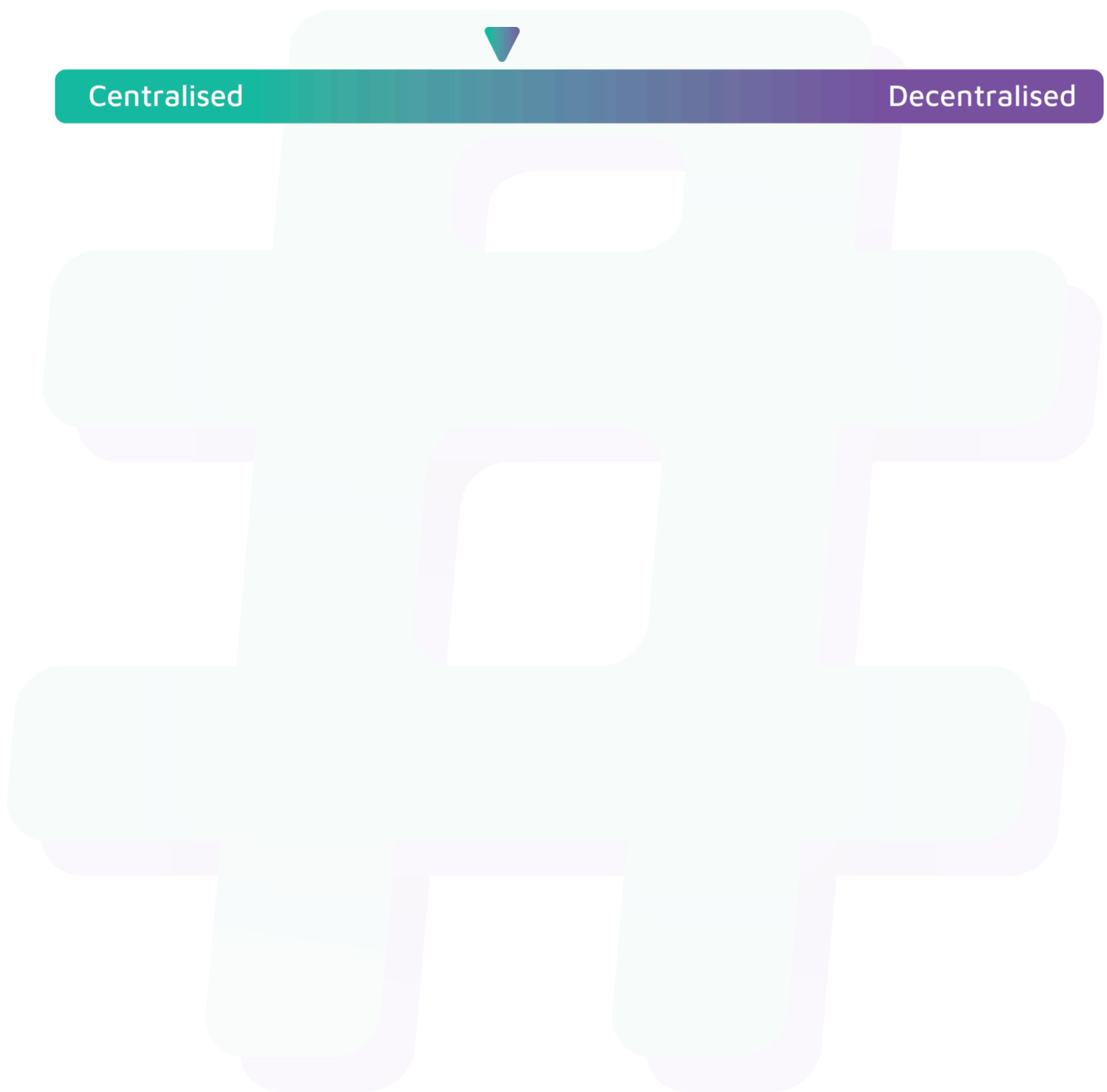
Status

Acknowledged

Centralisation

The Intiva Health values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Conclusion

After Hashlock's analysis, the Intiva Health project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.



Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.